



รายงาน

วิชา Internet of Things Fundamentals and Embedded Systems

เรื่อง การพัฒนาแกนกลหุ่นยนต์สำหรับการหยิบจับวัตถุพร้อมแผงควบคุมแบบ IoT

ผู้จัดทำ

67050334 นาย ปณณวัฒน์ เลิศโกเมนกุล

67050473 นาย รัชชานนท์ เหล่าสัมพันธ์

อาจารย์ประจำรายวิชา

รศ.ธีรวัฒน์ ประกอบผล

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา

05506271 Internet of Things Fundamentals and Embedded Systems

หลักสูตรวิทยาศาสตรบัณฑิต

สาขาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง (สจล.)

ประจำภาคการศึกษาที่ 2 ปีการศึกษา 2568

สารบัญ

บทที่1	1
บทนำ.....	1
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์ของการศึกษา	1
1.3 ประโยชน์ที่คาดว่าจะได้รับ.....	2
บทที่ 2	3
เอกสารและงานวิจัยที่เกี่ยวข้อง.....	3
2.1 แขนกลหุ่นยนต์ (ROBOT ARM)	3
2.2 เซอร์โวมอเตอร์ (SERVO MOTOR)	4
2.3 ไมโครคอนโทรลเลอร์ ESP32.....	5
2.4 การควบคุมอุปกรณ์ผ่าน WEB INTERFACE.....	6
2.6 ระบบบันทึกและเล่นซ้ำคำสั่ง (RECORD AND REPLAY)	7
2.7 งานวิจัยหรือโครงการที่เกี่ยวข้อง	7
บทที่ 3	9
ขั้นตอนและวิธีการดำเนินงาน	9
บทที่ 4	34
ผลการดำเนินงาน.....	34
4.1 ภาพรวมของระบบที่พัฒนา	34
4.2 โครงสร้างของระบบที่พัฒนา	34

4.3 โหมดการทำงานของระบบ	35
4.3.1 Hardware Control Mode	35
4.3.2 WiFi Control Mode.....	36
.....	36
4.3.3 Disabled Mode	37
4.4 ผลการทดสอบการทำงานของระบบ.....	37
4.5 สรุปผลการดำเนินงาน.....	37
บทที่ 5	38
สรุปและอภิปรายผลการดำเนินงาน.....	38
5.1 สรุปผลการดำเนินงาน.....	38
5.2 อภิปรายผลการดำเนินงาน	39
5.3 ข้อเสนอแนะและแนวทางการพัฒนาในอนาคต.....	39
บรรณานุกรม.....	40

บทที่1

บทนำ

1.1 ที่มาและความสำคัญ

ในกระบวนการทำงานหลายประเภท เช่น การหยิบจับวัตถุ การจัดเรียงสินค้า หรือการเคลื่อนย้ายชิ้นงานในโรงงาน มักต้องใช้แรงงานมนุษย์ในการทำงานซ้ำ ๆ ซึ่งอาจก่อให้เกิดปัญหา เช่น ความล้า ความผิดพลาดจากมนุษย์ และประสิทธิภาพในการทำงานที่ไม่สม่ำเสมอ นอกจากนี้ งานบางประเภทอาจต้องการความแม่นยำหรือการควบคุมที่ต่อเนื่องซึ่งมนุษย์ทำได้ยาก

ดังนั้นจึงมีความจำเป็นในการพัฒนา แขนกลหุ่นยนต์ (RobotArm) เพื่อช่วยในการหยิบจับและเคลื่อนย้ายวัตถุโดยอัตโนมัติ ซึ่งสามารถช่วยเพิ่มความแม่นยำ ลดภาระของมนุษย์ และเพิ่มประสิทธิภาพในการทำงาน โครงการนี้จึงมีเป้าหมายในการออกแบบและพัฒนา ระบบแขนกลหุ่นยนต์ที่สามารถควบคุมการเคลื่อนที่และการหยิบจับวัตถุได้ โดยใช้ไมโครคอนโทรลเลอร์และเซอร์โวมอเตอร์ เพื่อจำลองการทำงานของแขนกลในระบบอัตโนมัติ

1.2 วัตถุประสงค์ของการศึกษา

1. เพื่อศึกษาและปฏิบัติการเขียนโปรแกรมควบคุมไมโครคอนโทรลเลอร์ ESP32-S3 ด้วยภาษา MicroPython ในการสั่งงานแขนหุ่นยนต์ทั้ง 4 แกน
2. ใช้ module joystick ใช้ในการควบคุมทิศทางการหมุนของ servo ในแต่ละแกน
3. ศึกษาว่าหลักการที่ทำให้แขนหุ่นยนต์สามารถยับไปในทิศทางที่เรากำหนดด้วยการบังคับเองในรอบแรก และสามารถทำให้แขนหุ่นยนต์ยับตามที่เรายับเหมือนตอนแรกไปเรื่อยๆได้อย่างไร
4. สร้างหน้า web เพื่อให้โทรศัพท์สามารถควบคุมการทำงานจากระยะไกลได้

1.3ประโยชน์ที่คาดว่าจะได้รับ

1. ได้เรียนรู้หลักการออกแบบและพัฒนาแขนกลหุ่นยนต์ (Robot Arm) รวมถึงโครงสร้างทางกลและการทำงานของระบบหุ่นยนต์
2. ได้เข้าใจการควบคุม **Servo Motor** สำหรับการเคลื่อนที่ของข้อต่อแขนกลในระบบหุ่นยนต์
3. ได้พัฒนาทักษะการเขียนโปรแกรมและการใช้งาน **Microcontroller** เพื่อควบคุมอุปกรณ์อิเล็กทรอนิกส์
4. ได้เรียนรู้การพัฒนาาระบบควบคุมอุปกรณ์ผ่าน **Web Interface** และ **WiFi (Access Point)** สำหรับการควบคุมระยะไกล
5. สามารถนำความรู้ที่ได้ไปประยุกต์ใช้กับงานด้าน **Robotics, IoT และระบบอัตโนมัติ (Automation)** ในอนาคต

บทที่ 2

เอกสารและงานวิจัยที่เกี่ยวข้อง

2.1 แขนกลหุ่นยนต์ (Robot Arm)



แขนกลหุ่นยนต์ (Robot Arm) เป็นอุปกรณ์หุ่นยนต์ที่ถูกออกแบบมาให้มีลักษณะและการเคลื่อนไหวคล้ายกับแขนของมนุษย์ โดยมีหน้าที่หลักในการหยิบจับ เคลื่อนย้าย หรือจัดเรียงวัตถุ แขนกลมักถูกนำมาใช้ในงานอุตสาหกรรม เช่น การประกอบชิ้นส่วน การจัดเรียงสินค้า (Pick and Place) และการทำงานที่ต้องการความแม่นยำสูง

โครงสร้างของแขนกลหุ่นยนต์โดยทั่วไปประกอบด้วยส่วนสำคัญดังนี้

- **Base** เป็นฐานของแขนกล ทำหน้าที่รองรับโครงสร้างทั้งหมด
- **Link** เป็นส่วนที่เชื่อมต่อระหว่างข้อต่อต่าง ๆ
- **Joint** เป็นข้อต่อที่ทำให้แขนกลสามารถเคลื่อนที่ได้

- **End Effector** เป็นส่วนปลายของแขนกล เช่น ตัวหนีบ (Gripper) สำหรับหยิบจับวัตถุ

แขนกลหุ่นยนต์สามารถควบคุมการเคลื่อนที่ได้ด้วยมอเตอร์หลายประเภท เช่น Servo Motor หรือ Stepper Motor ซึ่งช่วยให้สามารถควบคุมตำแหน่งและทิศทางการเคลื่อนที่ได้อย่างแม่นยำ

2.2 เซอร์โวมอเตอร์ (Servo Motor)

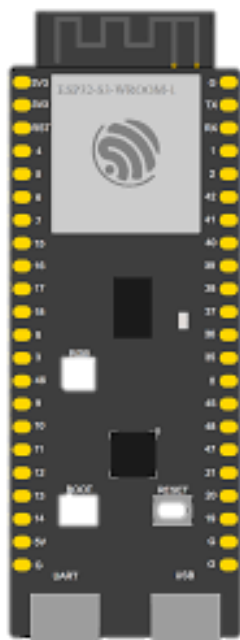


Servo Motor เป็นมอเตอร์ชนิดหนึ่งที่สามารถควบคุมตำแหน่ง ความเร็ว และทิศทางการหมุนได้อย่างแม่นยำ โดยภายในเซอร์โวมอเตอร์จะประกอบด้วยมอเตอร์ไฟฟ้า วงจรควบคุม และตัวตรวจวัดตำแหน่ง (Feedback System)

การควบคุมเซอร์โวมอเตอร์มักใช้สัญญาณ PWM (Pulse Width Modulation) ซึ่งเป็นสัญญาณดิจิทัลที่มีการปรับความกว้างของพัลส์เพื่อกำหนดตำแหน่งของมอเตอร์ ตัวอย่างเช่น สัญญาณ PWM ที่มีความกว้างต่างกันจะทำให้มอเตอร์หมุนไปยังตำแหน่งที่แตกต่างกัน

เซอร์โวมอเตอร์ถูกนำมาใช้อย่างแพร่หลายในงานด้านหุ่นยนต์ เนื่องจากสามารถควบคุมตำแหน่งได้อย่างแม่นยำ และตอบสนองต่อคำสั่งได้อย่างรวดเร็ว

2.3 ไมโครคอนโทรลเลอร์ ESP32



ESP32 เป็นไมโครคอนโทรลเลอร์ที่พัฒนาโดยบริษัท Espressif Systems ซึ่งได้รับความนิยมอย่างมากในงานด้านระบบสมองกลฝังตัว (Embedded Systems) และ Internet of Things (IoT)

คุณสมบัติสำคัญของ ESP32 ได้แก่

- มีหน่วยประมวลผลแบบ Dual-core
- รองรับการเชื่อมต่อ WiFi และ Bluetooth
- มีขา GPIO สำหรับเชื่อมต่อกับอุปกรณ์ภายนอก
- รองรับการเขียนโปรแกรมด้วยภาษา Python (MicroPython) หรือ C/C++

ESP32 จึงเหมาะสำหรับการพัฒนาโครงการที่ต้องการควบคุมอุปกรณ์อิเล็กทรอนิกส์และเชื่อมต่อเครือข่ายอินเทอร์เน็ต เช่น ระบบควบคุมหุ่นยนต์ผ่านเครือข่ายไร้สาย

2.4 การควบคุมอุปกรณ์ผ่าน Web Interface

การควบคุมอุปกรณ์ผ่าน Web Interface เป็นวิธีที่ช่วยให้ผู้ใช้งานสามารถสั่งงานอุปกรณ์ได้ผ่านเว็บเบราว์เซอร์โดยไม่ต้องติดตั้งโปรแกรมเพิ่มเติม

ในระบบนี้ ไมโครคอนโทรลเลอร์จะทำหน้าที่เป็น **Web Server** ที่คอยรับคำสั่งจากผู้ใช้งานผ่านโปรโตคอล HTTP (Hypertext Transfer Protocol) เมื่อผู้ใช้กดปุ่มควบคุมบนหน้าเว็บ ระบบจะส่งคำสั่งไปยังไมโครคอนโทรลเลอร์ ซึ่งจะนำคำสั่งนั้นไปควบคุมอุปกรณ์ต่าง ๆ เช่น มอเตอร์หรือเซนเซอร์

การใช้ Web Interface ทำให้ระบบสามารถใช้งานได้สะดวกและสามารถควบคุมได้จากอุปกรณ์หลายประเภท เช่น คอมพิวเตอร์ แท็บเล็ต หรือสมาร์ทโฟน

2.5 เครือข่าย WiFi และ Access Point



<https://devadiy.com/wp-content/uploads/e 1>

WiFi เป็นเทคโนโลยีเครือข่ายไร้สายที่ใช้ในการเชื่อมต่ออุปกรณ์ต่าง ๆ เข้าด้วยกันผ่านคลื่นวิทยุ ในไมโครคอนโทรลเลอร์ ESP32 สามารถทำงานได้ทั้งในโหมด **Station Mode** และ **Access Point Mode**

ในโหมด **Access Point (AP Mode)** ESP32 จะทำหน้าที่เป็นจุดกระจายสัญญาณ WiFi ทำให้อุปกรณ์อื่นสามารถเชื่อมต่อเข้ามาได้โดยตรง ซึ่งเหมาะสำหรับการพัฒนาโครงการที่ต้องการสร้างเครือข่ายเฉพาะเพื่อควบคุมอุปกรณ์ เช่น ระบบควบคุมแขนกลหุ่นยนต์ผ่านเว็บเบราว์เซอร์

2.6 ระบบบันทึกและเล่นซ้ำคำสั่ง (Record and Replay)

ระบบบันทึกและเล่นซ้ำคำสั่ง (Record and Replay) เป็นเทคนิคที่ใช้ในการบันทึกลำดับการเคลื่อนที่ของหุ่นยนต์ และสามารถนำข้อมูลที่บันทึกไว้มาทำงานซ้ำได้ในภายหลัง

กระบวนการทำงานของระบบ Record and Replay มีขั้นตอนดังนี้

1. บันทึกคำสั่งหรือการเคลื่อนที่ของอุปกรณ์
2. จัดเก็บข้อมูลไว้ในหน่วยความจำหรือไฟล์
3. อ่านข้อมูลที่บันทึกไว้และส่งคำสั่งไปยังอุปกรณ์ตามลำดับเวลาเดิม

ระบบนี้ช่วยให้หุ่นยนต์สามารถทำงานซ้ำได้โดยไม่ต้องควบคุมใหม่ทุกครั้ง และสามารถนำไปใช้ในงานอุตสาหกรรมที่ต้องทำงานแบบเดิมซ้ำ ๆ

2.7 งานวิจัยหรือโครงการที่เกี่ยวข้อง

จากการศึกษางานวิจัยและโครงการที่เกี่ยวข้อง พบว่ามีการพัฒนาแขนกลหุ่นยนต์ในหลากหลายรูปแบบ เช่น

งานวิจัยเกี่ยวกับการพัฒนาแขนกลหุ่นยนต์ที่ใช้ **Servo Motor** ในการควบคุมการเคลื่อนที่ของข้อต่อ โดยใช้ไมโครคอนโทรลเลอร์ในการประมวลผลและควบคุมตำแหน่งของแขนกล ทำให้สามารถหยิบจับและเคลื่อนย้ายวัตถุได้อย่างแม่นยำ

นอกจากนี้ยังมีการพัฒนาโครงการที่ใช้ ESP32 เป็น **Web Server** เพื่อควบคุมการทำงานของอุปกรณ์ผ่านเครือข่าย WiFi ผู้ใช้งานสามารถสั่งงานอุปกรณ์ผ่านหน้าเว็บได้แบบเรียลไทม์

จากการศึกษางานที่เกี่ยวข้องดังกล่าว แสดงให้เห็นว่าการนำไมโครคอนโทรลเลอร์ เซอร์โวมอเตอร์ และระบบเครือข่ายไร้สายมาประยุกต์ใช้ร่วมกันสามารถพัฒนาระบบแขนกลหุ่นยนต์ที่ควบคุมได้ผ่านเว็บและสามารถบันทึกการเคลื่อนที่เพื่อนำมาใช้งานซ้ำได้

ลำดับ	งานวิจัย / ผู้พัฒนา	เทคโนโลยีที่ใช้	วิธีการควบคุม	ความสามารถของระบบ
1	Design and Implementation of Robotic Arm using Arduino	Arduino, Servo Motor	ปุ่มควบคุม (Push Button)	ควบคุมการหมุนของแขนกลเพื่อหยิบจับวัตถุ
2	IoT Based Robotic Arm using ESP32	ESP32, WiFi, Servo Motor	Mobile Application	ควบคุมแขนกลผ่านเครือข่ายอินเทอร์เน็ต
3	Web-Based Robotic Arm Control	ESP32, Web Server, WiFi	Web Interface	ควบคุมแขนกลผ่านเว็บเบราว์เซอร์แบบเรียลไทม์
4	Robotic Arm with Motion Recording	Microcontroller, Servo Motor	Controller / Joystick	บันทึกและเล่นซ้ำการเคลื่อนที่ของแขนกล
5	โครงการนี้: Robot Arm Control System	ESP32, Servo Motor, WiFi, Web Interface, Joystick	Joystick และ Web Browser	ควบคุมแขนกลผ่านฮาร์ดแวร์และ Web Interface พร้อมระบบบันทึกและเล่นซ้ำการเคลื่อนที่ (Record & Replay)

เปรียบเทียบงานวิจัยที่เกี่ยวข้อง

บทที่ 3

ขั้นตอนและวิธีการดำเนินงาน

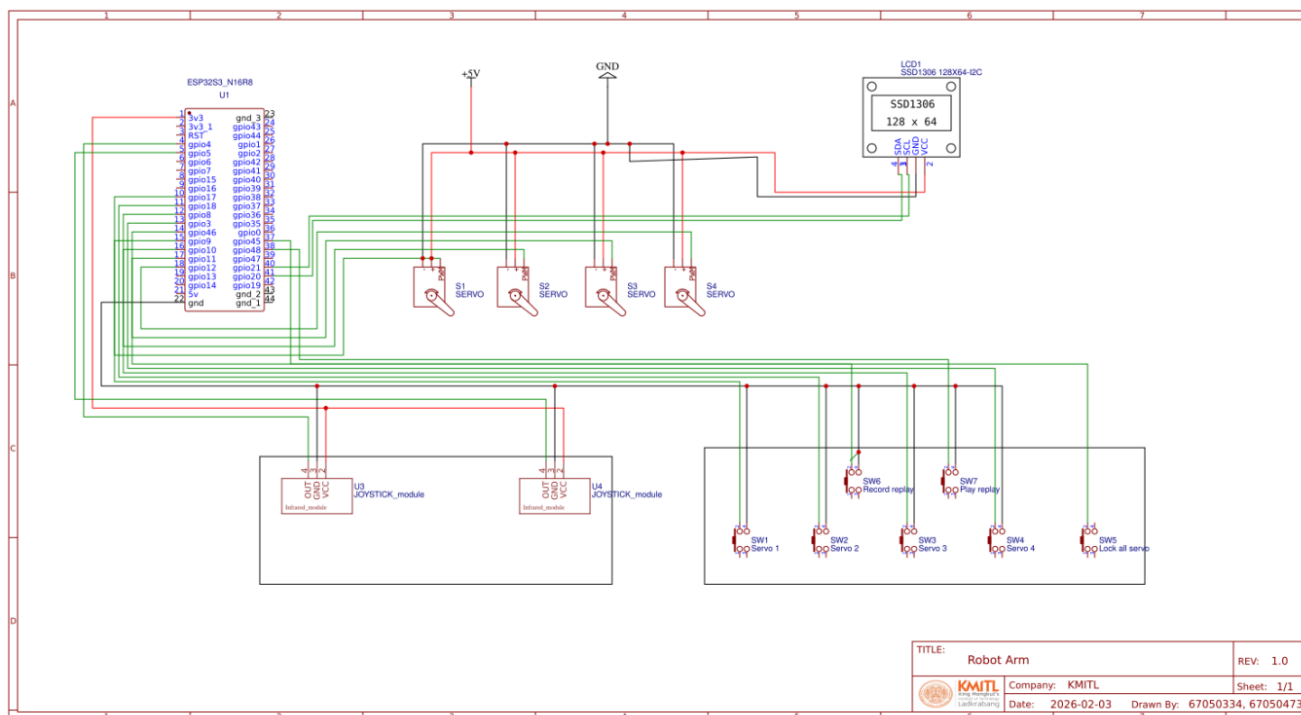
3.1 รายการอุปกรณ์

อุปกรณ์	จำนวน
ESP32-S3	1
Module Joystick	2
Bread Board	2
สายไฟ	50
ปุ่มเล็ก	5
ปุ่มใหญ่	2
จอ OLED	1
Servo MG90S	4
ชิ้นส่วนแขนหุ่นยนต์	13
ล้อ	50

3.2การต่อวงจร

อุปกรณ์	Pin	หน้าที่
Servo Motor 1 (MG90S)	15	มอเตอร์แกนที่ 1
Servo Motor 2 (MG90S)	16	มอเตอร์แกนที่ 2
Servo Motor 3 (MG90S)	17	มอเตอร์แกนที่ 3
Servo Motor 4 (MG90S)	18	มอเตอร์แกนที่ 4 ตัวหนีบ
Joystick 1 แกน X	4	ควบคุม Servo 2/3
Joystick 2 แกน X	5	ควบคุม Servo 1
OLED Display (SCL)	40	สัญญาณ Clock ของหน้าจอ
OLED Display (SDA)	41	สัญญาณ Data ของหน้าจอ
ปุ่มเล็ก 1	Pin 10 (Input Pull-up)	Lock/Unlock Servo 1
ปุ่มเล็ก 2	Pin 11 (Input Pull-up)	เลือกควบคุม Servo 2
ปุ่มเล็ก 3	Pin 12 (Input Pull-up)	เลือกควบคุม Servo 3
ปุ่มเล็ก 4	Pin 13 (Input Pull-up)	เปิด/ปิด ตัวหนีบ
ปุ่มเล็ก 5	Pin 14 (Input Pull-up)	Global Lock หยุดทุกแกน
ปุ่มใหญ่ 1	Pin 48 (Input Pull-up)	เริ่ม/หยุด บันทึกท่าทาง
ปุ่มใหญ่ 2	Pin 47 (Input Pull-up)	เล่นซ้ำท่าทางที่บันทึกไว้
Boot button	Pin 0 (Input Pull-up)	เปลี่ยน Mode (Physical/WiFi/Off)

schematics



3.3 หลักการทำงาน

1. การรับสัญญาณอินพุตจาก Joystick

Module Joystick ทำงานพื้นฐานด้วยตัวต้านทานปรับค่าได้ เมื่อผู้ใช้งานโยกก้านจอยสติ๊ก แรงดันไฟฟ้า ที่ส่งออกมาจะเกิดการเปลี่ยนแปลงในรูปแบบของ สัญญาณ Analog (ระหว่าง 0 ถึง 3.3V) ไมโครคอนโทรลเลอร์ (ESP32S3) ไม่สามารถประมวลผลค่า Analog ได้โดยตรง จึงต้องส่งสัญญาณนี้ผ่าน โมดูล ADC (Analog-to-Digital Converter) เพื่อแปลงขนาดของแรงดันไฟฟ้าให้เป็น ข้อมูลดิจิทัล (Digital Value) โดยในระบบนี้ใช้ความละเอียดแบบ 12-bit ค่าที่อ่านได้จึงอยู่ในช่วง 0 ถึง 4095

2. การประมวลผลและการตั้งค่าจุดศูนย์กลาง

2.1 เมื่อเปิดระบบจะทำการอ่านค่าตำแหน่งเริ่มต้นของจอยสติ๊กเพื่อบันทึกเป็น "จุดศูนย์กลาง" (Center Point: cx1, cx2) หลังจากนั้น เมื่อมีการใช้งานจริง ระบบจะคำนวณหาระยะการขยับ (Delta) ด้วย สมการ:

$$\text{Delta} = \text{Current_ADC_Value} - \text{Center_Point}$$

2.2 ระบบได้มีการออกแบบกลไก Deadzone ในโค้ดกำหนดค่าไว้ที่ 800 เพื่อแก้ปัญหาสัญญาณรบกวน หากค่าสัมบูรณ์ของ Delta มีค่าน้อยกว่า Deadzone ($\text{Delta} < 800$) ระบบจะถือว่าไม่มีการขยับ จอยสติ๊ก เพื่อป้องกันไม่ให้แขนหุ่นยนต์สั่นหรือขยับเองเมื่อผู้ใช้ปล่อยมือ

3. คณิตศาสตร์ของการสร้างสัญญาณ PWM

Servo ในโครงงานนี้ใช้ความถี่ในการทำงานที่ 50Hz ดังนั้นคาบเวลาของคลื่นสัญญาณ 1 รอบ สามารถคำนวณได้ดังนี้

$$T = 1 / f$$

$$T = 1 / 50 = 0.02 \text{ Seconds} = 20\text{ms}$$

ในการสั่งงาน ไมโครคอนโทรลเลอร์ใช้ฟังก์ชันการสร้างคลื่น PWM ด้วยความละเอียดระดับ 16-bit (duty_u16) ซึ่งหมายความว่าใน 1 คาบเวลา (20 ms) จะถูกแบ่งความละเอียดของการจ่ายไฟออกเป็น 0 ถึง 65535

การหาความกว้างของพัลส์ ที่ต้องการส่งไปยังเซอร์โว เพื่อคำนวณจุดหยุดนิ่งสามารถคำนวณได้จาก
สมการ:

$$\text{Duty_Value} = (t\text{-pulse} / T) \times 65535$$

ตัวอย่างการคำนวณจุดหยุดนิ่ง (Stop Position)

มาตรฐานของเซอร์โวมอเตอร์ จะหยุดนิ่งเมื่อได้รับความกว้างพัลส์ $t\text{-pulse} = 1.5\text{ms}$ เมื่อนำมา
แทนค่าในสมการจะได้:

$$\text{Duty_Value} = (1.5 / 20) \times 65535 \approx 4915$$

นี่คือที่มาของค่าคงที่ 4915 ในระบบที่ใช้ในการหยุดการเคลื่อนที่ของข้อต่อหุ่นยนต์

4. การแมปปิงคำสั่งขับเคลื่อนมอเตอร์

เซอร์โวมอเตอร์ที่ใช้เป็นชนิดหมุนต่อเนื่อง (Continuous Rotation) สัญญาณ PWM จึงไม่ได้
กำหนด "องศา" แต่เป็นการกำหนด "ทิศทางและความเร็ว" โครงการนี้ได้ออกแบบค่า Duty ไว้ 5 ระดับ
เพื่อควบคุมพลศาสตร์ของหุ่นยนต์ ดังตารางต่อไปนี้:

สถานะการทำงาน	ความกว้างพัลส์ โดยประมาณ (t-pulse)	ค่า Duty 16-bit	ผลลัพธ์ทางกลศาสตร์
FAST_CW	$\approx 1.0\text{ ms}$	3370	มอเตอร์หมุนตามเข็มนาฬิกา (เร็ว) / คลายกำมปู
SLOW_CW	$\approx 1.28\text{ ms}$	4200	มอเตอร์หมุนตามเข็มนาฬิกา (ช้า)
STOP_DUTY	1.5 ms	4915	มอเตอร์หยุดนิ่ง
SLOW_CCW	$\approx 1.7\text{ ms}$	5600	มอเตอร์หมุนทวนเข็มนาฬิกา (ช้า)
FAST_CCW	$\approx 2.0\text{ ms}$	6700	มอเตอร์หมุนทวนเข็มนาฬิกา (เร็ว) / หนีบกามปู

5. กลไกการบันทึกข้อมูลและการเล่นซ้ำ

5.1 กระบวนการบันทึกข้อมูล (Data Logging / Record Mode)

การบันทึกข้อมูลไม่ได้ใช้วิธีการบันทึกค่าตลอดเวลา (Continuous Logging) ซึ่งจะทำให้สิ้นเปลืองพื้นที่หน่วยความจำและทำให้ระบบทำงานช้าลง แต่ซอฟต์แวร์ได้ถูกออกแบบมาให้ใช้หลักการ Event-Driven Logging การบันทึกเมื่อมีการเปลี่ยนแปลงสถานะ

ขั้นตอนการทำงานมีดังนี้:

1. **เริ่มต้นบันทึก:** เมื่อผู้ใช้กดปุ่ม Record ระบบจะล้างข้อมูลเก่าในไฟล์ replay.txt ที่ (เขียนทับด้วยไฟล์ว่าง) และบันทึกเวลาเริ่มต้นระบบไว้ในตัวแปร start_time โดยใช้ฟังก์ชันอ่านค่ามิลลิวินาทีของฮาร์ดแวร์ time.ticks_ms()
2. **การตรวจจับการเปลี่ยนแปลง:** ในรูปการทำงานปกติ เมื่อมีการส่งคำสั่งขั้วมอเตอร์ ฟังก์ชัน write_log() จะถูกเรียกใช้งาน โปรแกรมจะนำค่า Duty_Value ล่าสุดไปเปรียบเทียบกับค่าในตัวแปร last_duty หากค่าเท่าเดิม โปรแกรมจะข้ามการทำงานไป เพื่อไม่ให้เกิดการเขียนข้อมูลซ้ำซ้อน
3. **การจัดเก็บข้อมูล:** หากมีการเปลี่ยนแปลงความเร็วหรือทิศทาง โปรแกรมจะคำนวณเวลาที่ผ่านไป จากจุดเริ่มต้นด้วยฟังก์ชัน time.ticks_diff() และนำข้อมูล 3 ส่วนประกอบเข้าด้วยกันในรูปแบบไฟล์ CSV ได้แก่:
 - Timestamp (t): เวลาที่เกิดคำสั่ง (หน่วยมิลลิวินาที)
 - Pin: หมายเลขขาพินของมอเตอร์ที่ถูกสั่งงาน (เช่น 15, 16, 17, 18)
 - Duty: ค่าสัญญาณ PWM ที่ส่งออกไป
4. **การเขียนลงไฟล์:** ข้อมูลจะถูกเขียนต่อท้าย ลงในไฟล์ replay.txt เช่น 1500,15,5600 (ความหมาย: ที่เวลา 1.5 วินาที สั่งให้พิน 15 จ่ายค่าพัลส์ 5600) จากนั้นระบบจะอัปเดตค่า last_duty เพื่อเตรียมรับการเปลี่ยนแปลงครั้งต่อไป

5.2 กระบวนการเล่นซ้ำ (Replay Mechanism)

เมื่อผู้ใช้กดปุ่ม Replay ระบบจะทำหน้าที่เป็นเสมือนเครื่องเล่นแผ่นเสียงที่อ่านข้อมูลจากไฟล์ replay.txt แล้วแปลงกลับเป็นสัญญาณไฟฟ้าทางกายภาพ โดยมีความท้าทายหลักคือ การชิงโครโนซ์เวลา เพื่อให้หุ่นยนต์ขยับด้วยความเร็วและจังหวะที่ตรงกับต้นฉบับทุกประการ

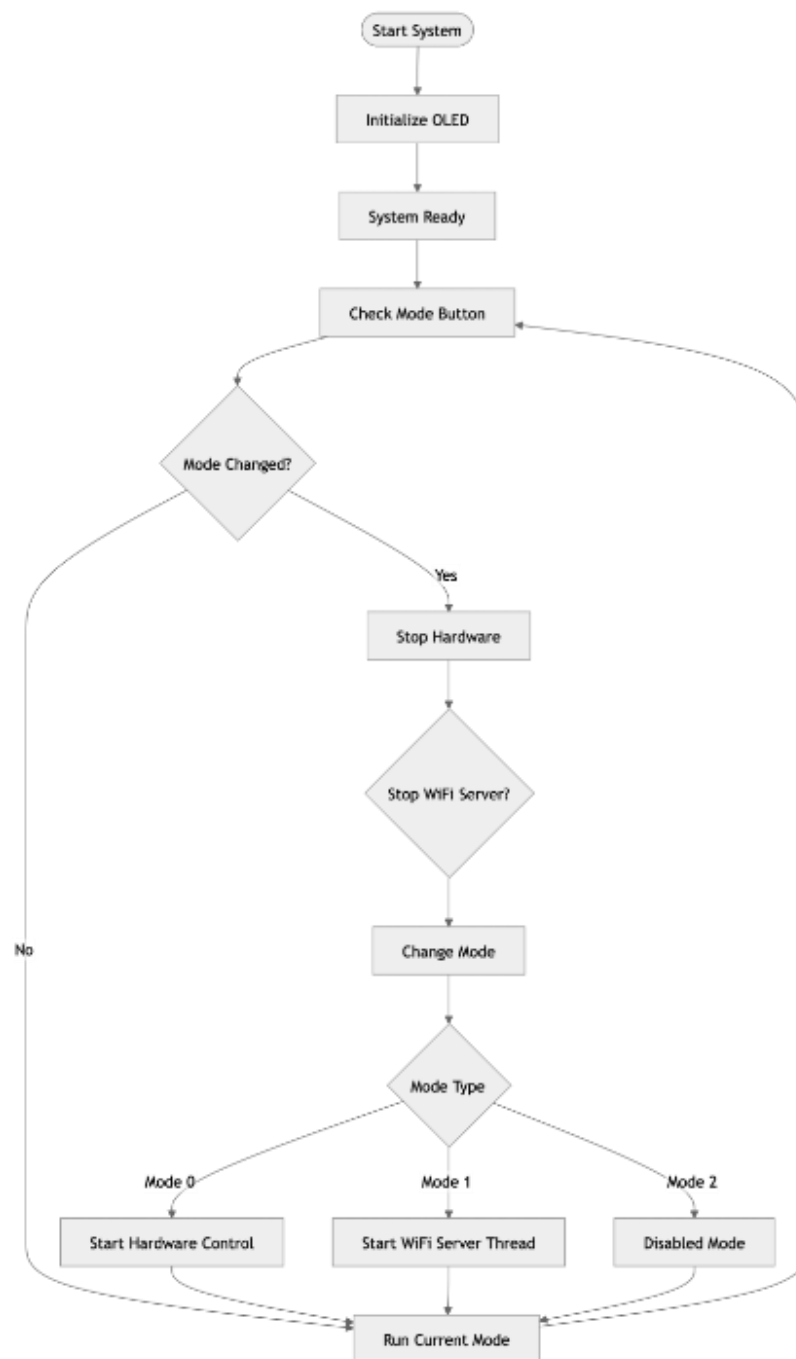
ขั้นตอนการทำงานมีดังนี้:

1. **การเตรียมความพร้อม:** ระบบจะเรียกใช้ฟังก์ชัน `stop_all()` เพื่อหยุดการเคลื่อนไหวของมอเตอร์ทุกตัวและรอ 200 มิลลิวินาที เพื่อป้องกันการกระชากของกระแสไฟฟ้า
2. **การตั้งค่าเวลาอ้างอิง:** โปรแกรมจะอ่านเวลาเริ่มต้นของการ Replay เก็บไว้ในตัวแปร `replay_start`
3. **การอ่านและแยกข้อมูล:** ระบบจะอ่านไฟล์ `replay.txt` ทีละ และใช้คำสั่ง `split(",")` เพื่อแยกข้อมูล Timestamp, Pin และ Duty ออกจากกัน
4. **การหน่วงเวลาให้ตรงจังหวะ:** โปรแกรมจะเข้าสู่ลูปเงื่อนไข

`t_current - t_replay_start < t_recorded`

ลูปนี้ จะทำงานวนซ้ำไปเรื่อยๆ จนกว่าเวลาจริงของระบบจะเดินมาถึงเวลาที่ระบุไว้ในไฟล์ข้อมูลแล้วไปที่บรรทัดต่อไป

Flowchart



3.5 Source code

project_iot_y2.py: โค้ดนี้คือ โปรแกรมหลัก ที่ทำหน้าที่เป็น "ตัวสลับโหมดการทำงาน" ของหุ่นยนต์ พร้อมกับแสดงผลสถานะออกทางหน้าจอ OLED

Mode 0 (Hardware): สั่งให้หุ่นยนต์อ่านค่าและรับคำสั่งจาก "จอยสติ๊ก"

Mode 1 (WiFi): สั่งเปิดเว็บเซิร์ฟเวอร์แบบเบื้องหลัง (Background Thread) เพื่อรอรับคำสั่งจาก "มือถือ/หน้าเว็บ"

Mode 2 (Disabled): โหมดหยุดพัก (Standby) ตัดการทำงานทุกอย่างเพื่อความปลอดภัย

```

project_iot_y2.py
1. from machine import Pin, I2C
2. from time import sleep
3. from ssd1306 import SSD1306_I2C
4. import mode
5. import Project_IoT_Y2_V2
6. import Project_IoT_Y2_V2_wifi
7. import test
8. import _thread
9.
10. # ===== OLED INIT =====
11. def init_oled():
12.     global i2c, oled
13.     try:
14.         i2c = I2C(1, scl=Pin(40), sda=Pin(41), freq=100000)
15.         oled = SSD1306_I2C(128, 64, i2c, addr=0x3c)
16.         print("OLED Ready")
17.     except Exception as e:
18.         print("OLED Error:", e)
19.         oled = None
20.
21. # ===== WIFI SERVER CONTROL =====
22. server_running = False
23. server_thread = None
24.
25. def run_wifi_server():
26.     """รัน WiFi server ในแบ็กกราวด์"""
27.     global server_running
28.     try:
29.         Project_IoT_Y2_V2_wifi.start_server()
30.     except Exception as e:
31.         print("WiFi Server Error:", e)
32.     finally:
33.         server_running = False
34.         print("WiFi Server Stopped")
35.
36. # ===== STARTUP =====
37. oled = None
38. i2c = None
39. init_oled()
40. current_mode = -1
41. print("=== SYSTEM READY ===")
42.
43. # ===== MAIN LOOP =====
44. while True:
45.     new_m = mode.update_mode()

```

```

project_iot_y2.py (main)
46.
47. # ----- Mode Change -----
48. if new_m != current_mode:
49.     current_mode = new_m
50.     print("Mode:", current_mode)
51.
52. # ===== หยุดทุกอย่างก่อน =====
53. try:
54.     Project_IoT_Y2_V2.stop_all()
55. except Exception as e:
56.     print("Error stopping hardware:", e)
57.
58. # หยุด WiFi server ถ้ากำลังทำงาน
59. if server_running:
60.     try:
61.         Project_IoT_Y2_V2_wifi.stop_all()
62.         server_running = False
63.         sleep(0.5) # รอให้ server หยุดจริง ๆ
64.         print("WiFi Server stopped")
65.     except Exception as e:
66.         print("Error stopping WiFi server:", e)
67.         server_running = False
68.
69. # ===== MODE 0: HARDWARE MODE =====
70. if current_mode == 0:
71.     try:
72.         Project_IoT_Y2_V2.start_all()
73.         if oled:
74.             test.display_mode0(oled)
75.             print("HARDWARE MODE started")
76.     except Exception as e:
77.         print("Error starting hardware mode:", e)
78.
79. # ===== MODE 1: WIFI MODE =====
80. elif current_mode == 1:
81.     try:
82.         if oled:
83.             test.display_mode1(oled)
84.             print("Entering WiFi Mode...")
85.             sleep(0.5)
86.
87.         # เริ่ม server ใน thread แยก
88.         server_running = True
89.         server_thread = _thread.start_new_thread(run_wifi_server, ())
90.         print("WiFi Server started in background thread")
91.     except Exception as e:
92.         print("Error starting WiFi mode:", e)
93.         server_running = False
94.
95. # ===== MODE 2: DISABLED =====
96. elif current_mode == 2:
97.     try:
98.         if oled:
99.             test.display_disabled(oled)
100.            print("DISABLED MODE")
101.        except Exception as e:
102.            print("Error in disabled mode:", e)
103.
104. # ----- RUN LOOP -----
105. try:
106.     if current_mode == 0:
107.         # HARDWARE MODE
108.         Project_IoT_Y2_V2.update()

```

```
project_iot_y2.py (main) == 1:
110.         # WIFI MODE - server running in background
111.         sleep(0.1)
112.     elif current_mode == 2:
113.         # DISABLED MODE
114.         sleep(0.1)
115. except Exception as e:
116.     print("Error in main loop:", e)
117.
118.     sleep(0.02)
119.
```


Project_IoT_Y2_V2.py: ระบบควบคุมแขนหุ่นยนต์ด้วย Module Joystick ซอฟต์แวร์ส่วนนี้ทำหน้าที่รับสัญญาณอินพุตจากผู้ใช้งานผ่านโมดูลจอยสติ๊กและปุ่มกดทางกายภาพ เพื่อแปลงเป็นคำสั่งขับเคลื่อนข้อต่อต่างๆ ของหุ่นยนต์ โดยมีฟังก์ชันการทำงานหลัก 5 ส่วน ดังนี้:

1. การควบคุมการเคลื่อนที่: อ่านค่าแอนะล็อก (ADC) จากจอยสติ๊ก 2 ตัว นำมาหักล้างกับค่า Deadzone เพื่อลดสัญญาณรบกวน แล้วแปลงเป็นความกว้างคลื่น PWM ส่งไปสั่งงาน Servo Motor ให้หมุนซ้าย/ขวา ช้าหรือเร็วตามระยะการโยก
2. ระบบสลับแกนควบคุม: เนื่องจากมีจอยสติ๊กจำกัด จึงออกแบบให้มีปุ่มกด (Pin 11, 12) สำหรับสลับการควบคุม ระหว่างข้อต่อที่ 2 (Pin 16) และข้อต่อที่ 3 (Pin 17) ทำให้สามารถควบคุมแขนได้ครบทุกแกน
3. ระบบความปลอดภัยและการล็อกแกน: มีปุ่ม Global Lock (Pin 14) สำหรับทำหน้าที่เป็นปุ่มหยุดฉุกเฉิน และปุ่มล็อกแกนฐาน (Pin 10) เพื่อป้องกันการหมุนโดยไม่ตั้งใจ
4. การควบคุมชุดหนีบจับ: มีปุ่มกด (Pin 13) สำหรับสั่งงานเปิดและปิดก้ามปู (Pin 18) โดยมีการหน่วงเวลา 350 มิลลิวินาทีในแต่ละจังหวะเพื่อให้มอเตอร์ทำงานได้สมบูรณ์ก่อนตัดกระแสไฟ (STOP_DUTY)
5. ระบบ Record & Replay: มีปุ่มกดเฉพาะสำหรับ เริ่ม/หยุด การบันทึก (Pin 48) และปุ่มเล่นซ้ำ (Pin 47) โดยอาศัยการจดบันทึกการเปลี่ยนแปลงของสัญญาณลงในไฟล์ replay.txt ตามที่ได้ออกแบบไว้

```

Project_IoT_Y2_V2.py
1. from machine import Pin, PWM, ADC
2. import time
3. import os
4.
5. # ===== CONFIG =====
6. PIN_SERVO_15 = 15
7. PINS_OTHERS = [16, 17, 18]
8.
9. STOP_DUTY = 4915
10. FAST_CW, FAST_CCW = 4200, 5600
11. SLOW_CW, SLOW_CCW = 3370, 6700
12. DEADZONE = 800
13.
14. GRIP = 6700
15. RELEASE = 3370
16.
17. RECORD_FILE = "replay.txt"
18.
19. # ===== HARDWARE =====
20. servo_15 = PWM(Pin(PIN_SERVO_15), freq=50)
21. servos = {p: PWM(Pin(p), freq=50) for p in PINS_OTHERS}
22.
23. joy1_x = ADC(Pin(4)); joy1_x.atten(ADC.ATTN_11DB)
24. joy2_x = ADC(Pin(5)); joy2_x.atten(ADC.ATTN_11DB)
25.
26. buttons = [Pin(p, Pin.IN, Pin.PULL_UP) for p in [10,11,12,13,14]]
27.
28. rec_btn = Pin(48, Pin.IN, Pin.PULL_UP)

```

```

Project_IoT_Y2_V2.py
29. replay_btn = Pin(47, Pin.IN, Pin.PULL_UP)
30.
31. # ===== STATE =====
32. selected_idx = 0
33. servo15_locked = False
34. global_lock = False
35. gripper_closed = False
36.
37. is_recording = False
38. start_time = 0
39. last_duty = {}
40.
41. cx1, cx2 = joy1_x.read(), joy2_x.read()
42.
43. # ===== BASIC FUNCTIONS =====
44.
45. def stop_all():
46.     servo_15.duty_u16(STOP_DUTY)
47.     for p in servos:
48.         servos[p].duty_u16(STOP_DUTY)
49.
50. def get_duty(pin, delta):
51.
52.     if abs(delta) < DEADZONE:
53.         return STOP_DUTY
54.
55.     if pin in [15,17]:
56.         return FAST_CW if delta > 0 else FAST_CCW
57.
58.     return SLOW_CW if delta > 0 else SLOW_CCW
59.
60.
61. # ===== RECORD =====
62.
63. def write_log(pin, duty):
64.
65.     if not is_recording:
66.         return
67.
68.     ts = time.ticks_diff(time.ticks_ms(), start_time)
69.
70.     if last_duty.get(pin) == duty:
71.         return
72.
73.     with open(RECORD_FILE, "a") as f:
74.         f.write("{}},{},{}\n".format(ts, pin, duty))
75.
76.     last_duty[pin] = duty
77.
78.
79. # ===== REPLAY =====
80.
81. def play_replay():
82.
83.     if RECORD_FILE not in os.listdir():
84.         print("No replay file")
85.         return
86.
87.     stop_all()
88.     time.sleep_ms(200)
89.
90.     with open(RECORD_FILE, "r") as f:
91.
92.         replay_start = time.ticks_ms()

```

Project_IoT_Y2_V2.py

```

93.
94.     for line in f:
95.
96.         parts = line.strip().split(",")
97.
98.         if len(parts) != 3:
99.             continue
100.
101.         t, pin, duty = map(int, parts)
102.
103.         while time.time() - replay_start < t:
104.             pass
105.
106.         if pin == 15:
107.             servo_15.duty_u16(duty)
108.
109.         elif pin in servos:
110.             servos[pin].duty_u16(duty)
111.
112.     stop_all()
113.     print("Replay Done")
114.
115.
116. # ===== GRIPPER =====
117.
118. def toggle_gripper():
119.     global gripper_closed
120.
121.     if gripper_closed:
122.
123.         duty = RELEASE
124.         servos[18].duty_u16(duty)
125.         write_log(18, duty)
126.
127.         time.sleep_ms(350)
128.
129.         servos[18].duty_u16(STOP_DUTY)
130.         write_log(18, STOP_DUTY)
131.
132.         gripper_closed = False
133.         print("Gripper Open")
134.
135.     else:
136.
137.         duty = GRIP
138.         servos[18].duty_u16(duty)
139.         write_log(18, duty)
140.
141.         time.sleep_ms(350)
142.
143.         servos[18].duty_u16(STOP_DUTY)
144.         write_log(18, STOP_DUTY)
145.
146.         gripper_closed = True
147.         print("Gripper Close")
148.
149.
150. # ===== START =====
151.
152. def start_all():
153.     print("[HARDWARE MODE]")
154.     stop_all()
155.
156.

```

```

Project_IoT_Y2_V2.py
157. # ===== UPDATE LOOP =====
158.
159. def update():
160.
161.     global selected_idx, servo15_locked, global_lock
162.     global is_recording, start_time
163.
164.     # ===== GLOBAL LOCK =====
165.     if buttons[4].value() == 0:
166.         global_lock = not global_lock
167.         stop_all()
168.         time.sleep_ms(300)
169.
170.     if global_lock:
171.         stop_all()
172.         return
173.
174.     # ===== SELECT SERVO =====
175.     for i in [1,2]:
176.         if buttons[i].value() == 0:
177.             selected_idx = i - 1
178.             time.sleep_ms(250)
179.
180.     # ===== JOYSTICK CONTROL S16/S17 =====
181.     current_pin = PINS_OTHERS[selected_idx]
182.
183.     dx1 = joy1_x.read() - cx1
184.
185.     duty = get_duty(current_pin, dx1)
186.
187.     servos[current_pin].duty_u16(duty)
188.     write_log(current_pin, duty)
189.
190.     # ===== LOCK SERVO 15 =====
191.     if buttons[0].value() == 0:
192.         servo15_locked = not servo15_locked
193.         time.sleep_ms(300)
194.
195.     # ===== GRIPPER =====
196.     if buttons[3].value() == 0:
197.         toggle_gripper()
198.         time.sleep_ms(300)
199.
200.     # ===== RECORD BUTTON =====
201.     if rec_btn.value() == 0:
202.
203.         is_recording = not is_recording
204.
205.         if is_recording:
206.
207.             with open(RECORD_FILE, "w") as f:
208.                 f.write("")
209.
210.             start_time = time.ticks_ms()
211.             last_duty.clear()
212.
213.             print("Recording Start")
214.
215.         else:
216.             print("Recording Stop")
217.
218.         time.sleep_ms(400)
219.
220.     # ===== REPLAY BUTTON =====

```

```
Project_IoT_Y2_V2.py
221.     if replay_btn.value() == 0:
222.         play_replay()
223.         time.sleep_ms(400)
224.
225.     # ===== JOYSTICK CONTROL SERVO15 =====
226.     if not servo15_locked:
227.
228.         dx2 = joy2_x.read() - cx2
229.
230.         duty = get_duty(15, dx2)
231.
232.         servo_15.duty_u16(duty)
233.         write_log(15, duty)
234.
235.     else:
236.         servo_15.duty_u16(STOP_DUTY)
237.
```

Project_IoT_Y2_V2_wifi.py: ระบบควบคุมแขนหุ่นยนต์ผ่าน WiFi Web Server

การทำงานเป็น 3 ส่วนหลัก ดังนี้:

1. การสร้างเครือข่าย (WiFi AP Setup): ระบบจะกระจายสัญญาณ WiFi ในชื่อ "ESP32-Robot" เพื่อให้อุปกรณ์ภายนอกเชื่อมต่อเข้ามายังวงแลนเดียวกัน
2. การให้บริการหน้าเว็บเพจ: เมื่อผู้ใช้งานเข้าถึงหมายเลข IP ของบอร์ด (เช่น 192.168.4.1) ระบบจะทำหน้าที่เป็นโฮสต์ ส่งไฟล์ HTML, CSS, และ JavaScript ที่เก็บไว้ในหน่วยความจำของบอร์ดไปแสดงผลเป็น "หน้าจอควบคุมเสมือน (Virtual Controller UI)" บนเบราว์เซอร์
3. การรับคำสั่งควบคุม (API Handler): เมื่อผู้ใช้กดปุ่มบนหน้าเว็บ เบราว์เซอร์จะส่ง HTTP Request (เช่น /s15_cw, /stop, /record) กลับมายังบอร์ด โปรแกรมจะทำหน้าที่แปลความหมาย (Parsing) ของ URL (Path) เหล่านั้นให้เป็นคำสั่งทางฮาร์ดแวร์เพื่อขับเคลื่อนมอเตอร์ บันทึกการทำงาน หรือเล่นซ้ำแบบเรียลไทม์

```

Project_IoT_Y2_V2_wifi.py
1. from machine import Pin, PWM
2. import time
3. import network
4. import socket
5. import os
6.
7. # =====
8. # WIFI AP SETUP
9. # =====
10. ap = network.WLAN(network.AP_IF)
11. ap.active(True)
12. ap.config(essid="ESP32-Robot", password="12345678", authmode=3)
13.
14. while not ap.active():
15.     pass
16.
17. print("AP IP:", ap.ifconfig()[0])
18.
19. # =====
20. # SERVO CONFIG (Continuous)
21. # =====
22. STOP_DUTY = 4915
23. FAST_CW   = 4200
24. FAST_CCW  = 5600
25. SLOW_CW   = 3370
26. SLOW_CCW  = 6700
27.
28. RECORD_FILE = "replay.txt"
29.
30. servos = {
31.     15: PWM(Pin(15), freq=50),
32.     16: PWM(Pin(16), freq=50),
33.     17: PWM(Pin(17), freq=50),
34.     18: PWM(Pin(18), freq=50)

```

```

35. }
36.
37. last_duty = {15: STOP_DUTY, 16: STOP_DUTY, 17: STOP_DUTY, 18: STOP_DUTY}
38. is_recording = False
39. start_time = 0
40.
41. # =====
42. # SERVO CONTROL
43. # =====
44. def set_servo(pin, duty):
45.     global last_duty
46.     duty = max(3000, min(7000, duty))
47.
48.     if pin in servos:
49.         servos[pin].duty_u16(duty)
50.
51.         if is_recording and duty != last_duty[pin]:
52.             write_log(pin, duty)
53.             last_duty[pin] = duty
54.
55. def stop_all():
56.     for pin in servos:
57.         servos[pin].duty_u16(STOP_DUTY)
58.     print("All Servos Stopped")
59.
60. # =====
61. # RECORD / REPLAY
62. # =====
63. def write_log(pin, duty):
64.     ts = time.time_diff(time.time_ms(), start_time)
65.     with open(RECORD_FILE, "a") as f:
66.         f.write("{}\n".format(ts, pin, duty))
67.
68. def play_replay():
69.     global is_recording
70.     if is_recording:
71.         return
72.
73.     try:
74.         stop_all()
75.         time.sleep(0.5)
76.
77.         with open(RECORD_FILE, "r") as f:
78.             replay_start = time.time_ms()
79.
80.             for line in f:
81.                 parts = line.strip().split(",")
82.                 if len(parts) != 3:
83.                     continue
84.
85.                 t = int(parts[0])
86.                 pin = int(parts[1])
87.                 duty = int(parts[2])
88.
89.                 while time.time_diff(time.time_ms(), replay_start) < t:
90.                     time.sleep_ms(1)
91.
92.                 if pin in servos:
93.                     servos[pin].duty_u16(duty)
94.
95.         stop_all()
96.         print("Replay Done")
97.
98.     except Exception as e:

```

```

Project_IoT_Y2_V2_wifi.py
99.     print("Replay Error:", e)
100.     stop_all()
101.
102. # =====
103. # API HANDLER
104. # =====
105. def handle_api(path):
106.     global is_recording, start_time
107.     print("PATH:", path)
108.
109.     try:
110.         # ===== SERVO COMMANDS =====
111.         if path == "/s15_cw": set_servo(15, SLOW_CW)
112.         elif path == "/s15_ccw": set_servo(15, SLOW_CCW)
113.
114.         elif path == "/s16_cw": set_servo(16, SLOW_CW)
115.         elif path == "/s16_ccw": set_servo(16, SLOW_CCW)
116.
117.         elif path == "/s17_cw": set_servo(17, FAST_CW)
118.         elif path == "/s17_ccw": set_servo(17, FAST_CCW)
119.
120.         elif path == "/s18_cw": set_servo(18, SLOW_CW)
121.         elif path == "/s18_ccw": set_servo(18, SLOW_CCW)
122.
123.         # ===== SYSTEM =====
124.         elif path == "/stop":
125.             stop_all()
126.
127.         elif path == "/record":
128.             is_recording = not is_recording
129.             if is_recording:
130.                 with open(RECORD_FILE, "w") as f:
131.                     f.write("")
132.                     start_time = time.time()
133.                     print("Recording Start")
134.             else:
135.                 print("Recording Stop")
136.
137.         elif path == "/replay":
138.             play_replay()
139.
140.     except Exception as e:
141.         print("API Error:", e)
142.
143. # =====
144. # STATIC FILE SENDER
145. # =====
146. def send_file(cl, filename, content_type):
147.     try:
148.         with open(filename, "rb") as f:
149.             cl.send("HTTP/1.1 200 OK\r\nContent-Type: {}\r\n\r\n".format(content_type))
150.             while True:
151.                 chunk = f.read(1024)
152.                 if not chunk:
153.                     break
154.                 cl.send(chunk)
155.     except:
156.         cl.send("HTTP/1.1 404 Not Found\r\n\r\nFile Not Found")
157.
158. # =====
159. # WEB SERVER
160. # =====
161. def start_server():
162.     s = socket.socket()

```



```

Project_IoT_Y2_V2_wifi.py
163. s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
164. s.bind(('0.0.0.0', 80))
165. s.listen(1)
166.
167. print("Server Ready")
168.
169. while True:
170.     cl, addr = s.accept()
171.     try:
172.         req = cl.recv(1024).decode()
173.         if not req:
174.             cl.close()
175.             continue
176.
177.         path = req.split(" ")[1]
178.         print("PATH:", path)
179.
180.         # ===== ROOT =====
181.         if path == "/" or path == "/index.html":
182.             send_file(cl, "dist/index.html", "text/html")
183.
184.         # ===== ASSETS =====
185.         elif path.startswith("/assets/"):
186.             filename = "dist" + path
187.             if filename.endswith(".js"):
188.                 send_file(cl, filename, "application/javascript")
189.             elif filename.endswith(".css"):
190.                 send_file(cl, filename, "text/css")
191.             elif filename.endswith(".svg"):
192.                 send_file(cl, filename, "image/svg+xml")
193.             else:
194.                 send_file(cl, filename, "application/octet-stream")
195.
196.         # ===== API =====
197.         elif path.startswith("/s") or path in ["/stop", "/record", "/replay"]:
198.             handle_api(path)
199.             cl.send("HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n\r\nOK")
200.
201.         else:
202.             cl.send("HTTP/1.1 404 Not Found\r\n\r\n")
203.
204.     except Exception as e:
205.         print("Server Error:", e)
206.
207.     cl.close()
208.

```

test.py: ระบบแสดงผลสถานะการทำงานผ่านจอ OLED

ซอฟต์แวร์ส่วนนี้ทำหน้าที่ควบคุมหน้าจอแสดงผลผ่านโปรโตคอลการสื่อสาร I2C เพื่อเป็นส่วนติดต่อผู้ใช้งาน ให้ทราบถึงสถานะการทำงานปัจจุบันของหุ่นยนต์ โดยแบ่งการแสดงผลตามโหมดต่างๆ ดังนี้:

1. การสร้างกราฟิกรูปภาพ (Bitmap Rendering): โปรแกรมใช้วิธีแปลงรูปภาพไอคอน (เช่น รูปจอยสติ๊ก และ รูปสัญญาณ WiFi) ให้อยู่ในรูปแบบอาร์เรย์ฐานสิบหก ขนาด 32x32 พิกเซล เพื่อให้ไมโครคอนโทรลเลอร์สามารถวาดรูปลงบนหน้าจอได้โดยไม่ต้องพึ่งพาไฟล์รูปภาพภายนอก
2. หน้าจอโหมดฮาร์ดแวร์ (Mode 0 - Physical Control): แสดงข้อความ "PHYSICAL CONTROL" พร้อมรูปไอคอนจอยสติ๊ก เพื่อบอกให้ผู้ใช้ทราบว่าหุ่นยนต์พร้อมรับคำสั่งจากปุ่มกดและจอยสติ๊กบนตัวเครื่อง
3. หน้าจอโหมดไร้สาย (Mode 1 - Mobile Control): แสดงข้อความ "MOBILE CONTROL" พร้อมรูปไอคอน WiFi เพื่อแสดงว่าระบบได้เปิดใช้งานเว็บเซิร์ฟเวอร์และพร้อมรับคำสั่งผ่านสมาร์ทโฟนแล้ว
4. หน้าจอโหมดพัก (Disabled Mode): แสดงข้อความ "SYSTEM DISABLED" พร้อมสัญลักษณ์กากบาท เพื่อเตือนว่าระบบปิดการทำงานเพื่อความปลอดภัย (Standby) และมอเตอร์ทุกตัวถูกตัดกระแสไฟแล้ว

```

test.py
1. from machine import Pin, I2C
2. from time import sleep
3. from ssd1306 import SSD1306_I2C
4. import mode
5.
6.
7. print("=== OLED Display with WiFi & Gamepad Icons ===\n")
8.
9. # Initialize I2C and OLED
10. try:
11.     i2c = I2C(1, scl=Pin(40), sda=Pin(41), freq=100000)
12.     oled = SSD1306_I2C(128, 64, i2c, addr=0x3c)
13.     print("✓ OLED initialized at 0x3c\n")
14. except:
15.     try:
16.         oled = SSD1306_I2C(128, 64, i2c, addr=0x3d)
17.         print("✓ OLED initialized at 0x3d\n")
18.     except Exception as e:
19.         print(f"X OLED Error: {e}")
20.
21. # WiFi Icon (32x32 pixels)
22. wifi_icon = [
23.     0x00, 0x00, 0x00, 0x00,
24.     0x00, 0x00, 0x00, 0x00,
25.     0x00, 0x00, 0x00, 0x00,
26.     0x00, 0x00, 0x00, 0x00,
27.     0x00, 0x00, 0x00, 0x00,
28.     0x00, 0x01, 0x80, 0x00,
29.     0x00, 0x1f, 0xfc, 0x00,
30.     0x00, 0xff, 0xff, 0x00,
31.     0x01, 0xff, 0xff, 0xc0,

```

test.py

```

32.     0x03, 0xf0, 0x0f, 0xe0,
33.     0x0f, 0xc0, 0x03, 0xf0,
34.     0x0f, 0x0f, 0xf0, 0xf0,
35.     0x06, 0x3f, 0xfc, 0x60,
36.     0x00, 0x7f, 0xff, 0x00,
37.     0x00, 0xfc, 0x3f, 0x80,
38.     0x00, 0xf0, 0x07, 0x00,
39.     0x00, 0x47, 0xe2, 0x00,
40.     0x00, 0x0f, 0xf8, 0x00,
41.     0x00, 0x1f, 0xf8, 0x00,
42.     0x00, 0x0e, 0x38, 0x00,
43.     0x00, 0x00, 0x10, 0x00,
44.     0x00, 0x01, 0xc0, 0x00,
45.     0x00, 0x03, 0xe0, 0x00,
46.     0x00, 0x07, 0xe0, 0x00,
47.     0x00, 0x07, 0xe0, 0x00,
48.     0x00, 0x07, 0xe0, 0x00,
49.     0x00, 0x03, 0xe0, 0x00,
50.     0x00, 0x01, 0xc0, 0x00,
51.     0x00, 0x00, 0x00, 0x00,
52.     0x00, 0x00, 0x00, 0x00,
53.     0x00, 0x00, 0x00, 0x00,
54.     0x00, 0x00, 0x00, 0x00
55. ]
56.
57. # Gamepad Icon (32x32 pixels)
58. gamepad_icon = [
59.     0x00, 0x00, 0x00, 0x00,
60.     0x00, 0x00, 0x00, 0x00,
61.     0x00, 0x00, 0x00, 0x00,
62.     0x00, 0x00, 0x00, 0x00,
63.     0x07, 0x80, 0x0f, 0x80,
64.     0x1f, 0xe0, 0x1f, 0xc0,
65.     0x3f, 0xf0, 0x3f, 0xe0,
66.     0x3f, 0xff, 0xff, 0xf0,
67.     0x7f, 0xff, 0xff, 0xf0,
68.     0x7e, 0xff, 0xf9, 0xf8,
69.     0xfe, 0xff, 0xf8, 0xf8,
70.     0xfc, 0xff, 0xf8, 0xf8,
71.     0xf8, 0x3f, 0xe2, 0x3c,
72.     0xfe, 0xff, 0xe6, 0x3c,
73.     0xfe, 0xff, 0xe0, 0x7c,
74.     0xfe, 0xff, 0xf8, 0xfc,
75.     0xff, 0xff, 0xf8, 0xfc,
76.     0xff, 0xff, 0xff, 0xfe,
77.     0xff, 0xff, 0xff, 0xfe,
78.     0xff, 0xff, 0xff, 0xfe,
79.     0xff, 0xff, 0xff, 0xfe,
80.     0xff, 0xff, 0xff, 0xfe,
81.     0xff, 0xfc, 0xff, 0xfe,
82.     0xff, 0xc0, 0x1f, 0xfe,
83.     0xff, 0x80, 0x0f, 0xfe,
84.     0xff, 0x00, 0x07, 0xfe,
85.     0xfe, 0x00, 0x03, 0xfe,
86.     0xfc, 0x00, 0x01, 0xfc,
87.     0xf8, 0x00, 0x00, 0x7c,
88.     0x00, 0x00, 0x00, 0x00,
89.     0x00, 0x00, 0x00, 0x00,
90.     0x00, 0x00, 0x00, 0x00
91. ]
92.
93. def draw_bitmap(oled, x, y, bitmap, width=32, height=32):
94.     """Draw a 32x32 bitmap on OLED display"""
95.     bytes_per_row = (width + 7) // 8

```

```

test.py
96.
97.     for row in range(height):
98.         for byte_idx in range(bytes_per_row):
99.             byte_val = bitmap[row * bytes_per_row + byte_idx]
100.            for bit in range(8):
101.                pixel_x = x + byte_idx * 8 + bit
102.                pixel_y = y + row
103.
104.                # Draw if pixel is within display bounds
105.                if pixel_x < 128 and pixel_y < 64:
106.                    if byte_val & (1 << (7 - bit)):
107.                        oled.pixel(pixel_x, pixel_y, 1)
108.
109. def display_mode0(oled):
110.     """Mode 0: Physical Control with Gamepad"""
111.     oled.fill(0)
112.
113.     # Draw gamepad icon on right side
114.     draw_bitmap(oled, 65, 8, gamepad_icon)
115.
116.     # Text on left
117.     oled.text('PHYSICAL', 0, 8)
118.     oled.text('CONTROL', 5, 20)
119.
120.     # Status bar at bottom
121.     oled.text('Mode: 0', 0, 50)
122.     oled.text('Active', 70, 50)
123.
124.     oled.show()
125.
126. def display_mode1(oled):
127.     """Mode 1: Mobile Control with WiFi"""
128.     oled.fill(0)
129.
130.     # Draw WiFi icon on right side
131.     draw_bitmap(oled, 75, 10, wifi_icon)
132.
133.     # Text on left
134.     oled.text('MOBILE', 10, 8)
135.     oled.text('CONTROL', 5, 20)
136.
137.     # Status bar at bottom
138.     oled.text('Mode: 1', 0, 50)
139.     oled.text('WiFi', 75, 50)
140.
141.     oled.show()
142.
143. def display_disabled(oled):
144.     """Disabled Mode"""
145.     oled.fill(0)
146.
147.     # Center text
148.     oled.text('SYSTEM', 20, 15)
149.     oled.text('DISABLED', 15, 30)
150.
151.     # Draw X mark
152.     for i in range(12):
153.         oled.pixel(75 + i, 12 + i, 1)
154.         oled.pixel(75 + i, 23 - i, 1)
155.
156.     # Status bar
157.     oled.text('Mode: X', 0, 50)
158.     oled.text('OFF', 75, 50)
159.

```

test.py

```
160.     oled.show()  
161.  
162. # Main loop  
163.
```

บทที่ 4

ผลการดำเนินงาน

4.1 ภาพรวมของระบบที่พัฒนา

โครงการนี้ได้พัฒนาระบบแขนกลหุ่นยนต์ที่สามารถควบคุมการเคลื่อนที่ของแขนกลได้ผ่าน 2 รูปแบบ ได้แก่ การควบคุมผ่านอุปกรณ์ฮาร์ดแวร์ (Joystick และปุ่มควบคุม) และการควบคุมผ่าน Web Interface โดยใช้ไมโครคอนโทรลเลอร์ ESP32 เป็นตัวควบคุมหลักของระบบ

ระบบประกอบด้วย Servo Motor จำนวน 4 ตัวสำหรับควบคุมข้อต่อของแขนกล โดยผู้ใช้งานสามารถควบคุมการเคลื่อนที่ของแขนกลเพื่อหยิบจับหรือเคลื่อนย้ายวัตถุได้ตามต้องการ นอกจากนี้ระบบยังมีฟังก์ชันบันทึกและเล่นซ้ำการเคลื่อนที่ของแขนกล (Record and Replay) เพื่อให้สามารถทำงานซ้ำได้โดยอัตโนมัติ

4.2 โครงสร้างของระบบที่พัฒนา

ระบบแขนกลหุ่นยนต์ที่พัฒนาขึ้นประกอบด้วยส่วนประกอบหลักดังนี้

1. ไมโครคอนโทรลเลอร์ ESP32 สำหรับควบคุมการทำงานของระบบ
2. Servo Motor จำนวน 4 ตัว สำหรับควบคุมข้อต่อของแขนกล
3. Joystick Module สำหรับควบคุมการเคลื่อนที่ของแขนกลในโหมดฮาร์ดแวร์
4. Push Button สำหรับเลือก Servo และสั่งงาน Record / Replay
5. OLED Display สำหรับแสดงสถานะของระบบ
6. Web Interface สำหรับควบคุมแขนกลผ่านเครือข่าย WiFi

4.3 โหมดการทำงานของระบบ

ระบบที่พัฒนาขึ้นสามารถทำงานได้ทั้งหมด 3 โหมด ได้แก่

4.3.1 Hardware Control Mode

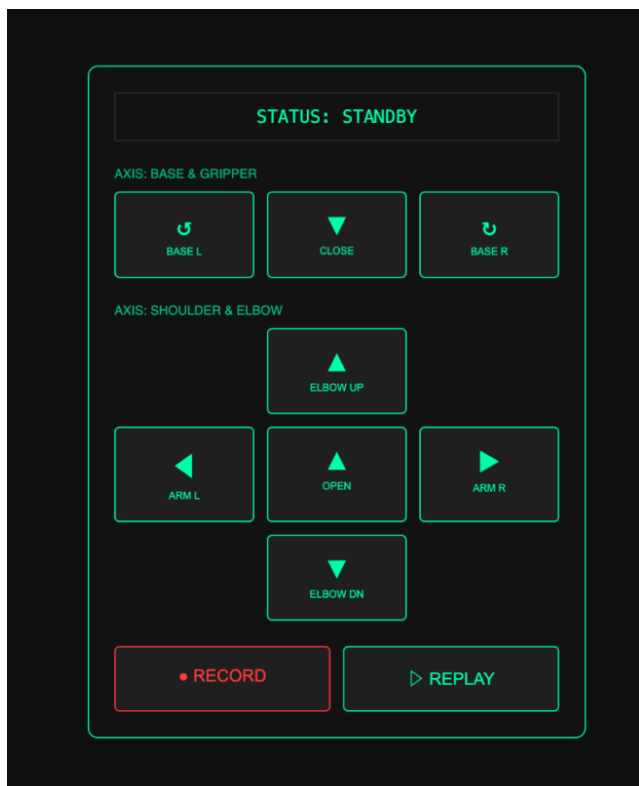


ในโหมดนี้ผู้ใช้งานสามารถควบคุมแขนกลผ่าน Joystick และปุ่มกด โดยผู้ใช้งานสามารถเลือก Servo Motor ที่ต้องการควบคุม และใช้ Joystick ในการควบคุมทิศทางการหมุนของ Servo Motor เพื่อเคลื่อนที่แขนกล

นอกจากนี้ยังมีปุ่มสำหรับ

- บันทึกการเคลื่อนที่ของแขนกล (Record)
- เล่นซ้ำการเคลื่อนที่ที่บันทึกไว้ (Replay)
- หยุดการทำงานของ Servo Motor

4.3.2 WiFi Control Mode



ในโหมดนี้ระบบจะทำการเปิด WiFi Access Point บน ESP32 เพื่อให้ผู้ใช้งานสามารถเชื่อมต่อกับระบบผ่านอุปกรณ์ต่าง ๆ เช่น คอมพิวเตอร์หรือสมาร์ทโฟน

เมื่อเชื่อมต่อกับเครือข่าย WiFi ของระบบแล้ว ผู้ใช้งานสามารถเปิด Web Browser และเข้าถึงหน้า

Web Interface เพื่อควบคุมการเคลื่อนที่ของแขนกล เช่น

- หมุน Servo Motor
- หยุดการทำงานของแขนกล
- บันทึกการเคลื่อนที่
- เล่นซ้ำการเคลื่อนที่

4.3.3 Disabled Mode

ในโหมดนี้ระบบจะหยุดการทำงานของ Servo Motor ทั้งหมด เพื่อป้องกันการเคลื่อนที่ของแขนกลโดยไม่ได้ตั้งใจ และช่วยเพิ่มความปลอดภัยในการใช้งาน

4.4 ผลการทดสอบการทำงานของระบบ

จากการทดสอบระบบแขนกลหุ่นยนต์ พบว่าระบบสามารถทำงานได้ตามวัตถุประสงค์ที่กำหนดไว้ โดยมีผลการทดสอบดังนี้

1. ระบบสามารถควบคุม Servo Motor ได้ครบทั้ง 4 ตัว
2. ผู้ใช้งานสามารถควบคุมแขนกลผ่าน Joystick ได้อย่างถูกต้อง
3. ระบบสามารถสร้าง WiFi Access Point และให้ผู้ใช้งานเชื่อมต่อเพื่อควบคุมผ่าน Web Interface ได้
4. ระบบสามารถบันทึกการเคลื่อนที่ของแขนกลและเล่นซ้ำการทำงานได้
5. ระบบสามารถสลับโหมดการทำงานระหว่าง Hardware Mode และ WiFi Mode ได้อย่างถูกต้อง

4.5 สรุปผลการดำเนินงาน

จากการพัฒนาและทดสอบระบบแขนกลหุ่นยนต์ พบว่าระบบสามารถทำงานได้ตามวัตถุประสงค์ที่ตั้งไว้ โดยสามารถควบคุมแขนกลผ่านทั้งอุปกรณ์ฮาร์ดแวร์และ Web Interface ผ่านเครือข่าย WiFi นอกจากนี้ยังมีระบบบันทึกและเล่นซ้ำการเคลื่อนที่ ซึ่งช่วยให้แขนกลสามารถทำงานซ้ำได้โดยอัตโนมัติ

ระบบที่พัฒนาขึ้นสามารถนำไปประยุกต์ใช้ในงานด้านระบบอัตโนมัติ หุ่นยนต์ หรือระบบ Internet of Things (IoT) ได้ในอนาคต

บทที่ 5

สรุปและอภิปรายผลการดำเนินงาน

5.1 สรุปผลการดำเนินงาน

โครงการการพัฒนาแขนกลหุ่นยนต์ (Robot Arm) สำหรับการหยิบจับและเคลื่อนย้ายวัตถุ ได้มีการออกแบบและพัฒนาระบบควบคุมโดยใช้ไมโครคอนโทรลเลอร์ **ESP32** เป็นตัวควบคุมหลักของระบบ โดยใช้ **Servo Motor** ในการควบคุมการเคลื่อนที่ของข้อต่อแขนกล เพื่อให้สามารถเคลื่อนที่และหยิบจับวัตถุได้ตามต้องการ ระบบที่พัฒนาขึ้นสามารถควบคุมการทำงานของแขนกลได้ 2 รูปแบบ ได้แก่ การควบคุมผ่านอุปกรณ์ฮาร์ดแวร์โดยใช้ **Joystick และปุ่มควบคุม** และการควบคุมผ่าน **Web Interface ผ่านเครือข่าย WiFi** ซึ่งช่วยเพิ่มความสะดวกในการใช้งานและสามารถควบคุมระบบได้จากอุปกรณ์ต่าง ๆ เช่น คอมพิวเตอร์หรือสมาร์ทโฟน

นอกจากนี้ ระบบยังมีฟังก์ชัน **Record and Replay** สำหรับบันทึกการเคลื่อนที่ของแขนกลและสามารถเล่นซ้ำการทำงานที่บันทึกไว้ได้ ซึ่งช่วยให้แขนกลสามารถทำงานซ้ำได้โดยอัตโนมัติ จากผลการทดสอบพบว่าระบบสามารถทำงานได้ตามวัตถุประสงค์ที่กำหนดไว้ โดยสามารถควบคุม Servo Motor ได้ครบทุกตัว ระบบ WiFi สามารถเชื่อมต่อและควบคุมผ่านหน้าเว็บได้อย่างถูกต้อง และระบบบันทึกและเล่นซ้ำการเคลื่อนที่ที่สามารถทำงานได้อย่างมีประสิทธิภาพ

5.2 อภิปรายผลการดำเนินงาน

จากการพัฒนาและทดสอบระบบแขนกลหุ่นยนต์ พบว่าการใช้ไมโครคอนโทรลเลอร์ ESP32 สามารถรองรับการควบคุม Servo Motor และการเชื่อมต่อเครือข่าย WiFi ได้อย่างมีประสิทธิภาพ ทำให้สามารถพัฒนาระบบควบคุมแขนกลผ่าน Web Interface ได้โดยไม่จำเป็นต้องใช้อุปกรณ์เพิ่มเติม การควบคุมแขนกลผ่าน Joystick ช่วยให้ผู้ใช้สามารถควบคุมการเคลื่อนที่ของแขนกลได้อย่างสะดวกและตอบสนองต่อคำสั่งได้รวดเร็ว ในขณะที่การควบคุมผ่าน Web Interface ช่วยให้สามารถควบคุมแขนกลได้จากระยะไกลผ่านเครือข่าย WiFi

อย่างไรก็ตาม ระบบยังมีข้อจำกัดบางประการ เช่น ความแม่นยำของการเคลื่อนที่ของแขนกลที่ขึ้นอยู่กับความละเอียดของ Servo Motor และข้อจำกัดของแรงบิดของ Servo Motor ที่ใช้ในโครงงาน ซึ่งอาจส่งผลกระทบต่อการทำงานของวัตถุที่มีน้ำหนักมาก

5.3 ข้อเสนอแนะและแนวทางการพัฒนาในอนาคต

เพื่อให้ระบบแขนกลหุ่นยนต์มีประสิทธิภาพมากยิ่งขึ้น สามารถพัฒนาเพิ่มเติมได้ในอนาคต ดังนี้

1. เพิ่มจำนวน Servo Motor หรือปรับปรุงโครงสร้างแขนกลเพื่อเพิ่มความยืดหยุ่นในการเคลื่อนที่
2. เพิ่มเซนเซอร์ เช่น เซนเซอร์ตรวจจับวัตถุ หรือกล้อง เพื่อให้แขนกลสามารถทำงานแบบอัตโนมัติได้
3. พัฒนา Web Interface ให้มีฟังก์ชันการควบคุมที่หลากหลายมากขึ้น เช่น การควบคุมแบบ Slider หรือการแสดงผลสถานะของแขนกลแบบเรียลไทม์
4. พัฒนาระบบให้สามารถเชื่อมต่อกับอินเทอร์เน็ต (IoT) เพื่อควบคุมแขนกลจากระยะไกลผ่านเครือข่ายอินเทอร์เน็ต

บรรณานุกรม

1. Benitez, V. H. (2020). *Design of an affordable IoT open-source robot arm for robotics education*. IEEE Access.
2. Espressif Systems. (2023). *ESP32 Series Datasheet*. Espressif Systems.
<https://www.espressif.com>
3. Kareemullah, H. (2021). *Design and implementation of robotic arm and control of moving via IoT with Arduino ESP32*. ResearchGate.
4. Thulasi, P. R., & Kumar, S. (2022). *Development of robotic arm control using Arduino controller*. International Journal of Engineering Research and Technology.
5. Telengre, H., & Patil, A. (2023). *ESP32 based robotic arm vehicle using IoT*. International Journal of Scientific Research and Development.
6. Monk, S. (2017). *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education.
7. Margolis, M. (2019). *Arduino Cookbook* (3rd ed.). O'Reilly Media.